

Rozmowa w Ideo – Flutter

Proste pytania i odpowiedzi

Adrian Rojek
11.09.2026, godz. 10:00
ul. Nad Przyrwą 13

Spis treści

1	Czy będą kazać kodować na żywo?	2
2	Powiedz o sobie w 30 sekund	2
3	Pytania miękkie / o pracę	2
4	Flutter i Dart – prosto	3
5	Stan aplikacji: Provider, Cubit, Bloc	4
6	REST – rozmowa z serwerem	4
7	Firebase	5
8	UI i design	5
9	Natywne Android / iOS	5
10	Wrzucanie do sklepów	6
11	Architektura i git	6
12	Twoje projekty – co mówić	6
12.1	DEVEJI	6
12.2	Fruit Tracker (Flutter, prywatne)	7
12.3	Water Reminder	7
12.4	Fitness Tracker	7
12.5	MacSpaceCleaner	7
12.6	Courier (Electron)	7
12.7	Bus (Laravel)	7
13	Trudne pytania – uczciwe odpowiedzi	7
14	Pytania behawioralne – szkielet	8
15	Gdyby jednak dali małe zadanie	8
16	Pytania, które Ty zadajesz Im	8
17	Checklista przed rozmową	9

1 Czy będą kazać kodować na żywo?

Raczej nie. Spotkanie trwa max 45 minut. Będą 2 osoby techniczne. Więcej czasu zajmie gadanie o Tobie, projektach i firmie niż siedzenie przy laptopie.

Przygotuj się na:

- pytania o Fluttera, Firebase, REST, stan aplikacji,
- opowiadanie o swoich apkach,
- ewentualnie krótkie „jak byś to zrobił” na tablicy / ustnie.

Jak będzie druga runda – dopiero wtedy częściej bywa zadanie albo live coding.

2 Powiedz o sobie w 30 sekund

Pytanie: Opowiedz coś o sobie.

Odpowiedź: Jestem studentem informatyki w Rzeszowie. Robię aplikacje mobilne – głównie Flutter i SwiftUI. W DEVEJI robiłem Fluttera z Firebase i wrzucałem apki do App Store i Google Play. Mam też własną apkę Flutter (owoce/warzywa, skan ML, SQLite) oraz apki iOS typu Water Reminder i Fitness Tracker.

Pytanie: Dlaczego Ideo?

Odpowiedź: Szukacie Fluttera, Firebase, REST, ładnego UI i kogoś kto ogarnia też natywne iOS/Android. To pasuje do tego, co już robiłem. Chcę robić realne projekty dla klientów w software house i uczyć się od zespołu. Rzeszów i hybryda mi pasują.

3 Pytania miękkie / o pracę

Pytanie: Jak wyglądała praca w DEVEJI?

Odpowiedź: Robiłem ekrany we Flutterze, łączyłem z Firebase, poprawiałem bugi, przygotowywałem buildy do sklepów. Jak coś było niejasne albo ryzykowne (np. review w App Store) – mówię wcześniej, nie na ostatnią chwilę.

Pytanie: Co robisz, gdy wymagania są niejasne?

Odpowiedź: Dopytuję. Ustalam najprostsze MVP. Wolę potwierdzić założenie niż zgadywać przez dwa dni.

Pytanie: Co gdy utkniesz na błędzie?

Odpowiedź: Odtwarzam problem na minimum. Patrzą logi. Odcinam warstwy: UI / stan / sieć. Jak blokuję innych – mówię szybko i proponuję plan B.

Pytanie: Remote, hybryda czy biuro?

Odpowiedź: Hybryda. Na start chętnie bywam w biurze, potem według projektu.

Pytanie: Masz apki w sklepach?

Odpowiedź: Tak – przy DEVEJI brałem udział w publikacji iOS i Androida. Mam też własne projekty iOS. O statusie Fruit Trackera mów tylko zgodnie z prawdą.

Pytanie: Dasz radę też przy innych technologiach?

Odpowiedź: Tak. Flutter będzie główny, ale znam SwiftUI, trochę Kotliną, robiłem też Electron/Node i proste strony. Oferta Ideo właśnie to przewiduje.

Pytanie: Studia + praca?

Odpowiedź: Robię magistra. Potrafię łączyć – sesję i wolne komunikuję z wyprzedzeniem.

4 Flutter i Dart – prosto

Pytanie: Co to Flutter?

Odpowiedź: Narzędzie do robienia aplikacji mobilnych. Piszesz w Dart. Jedna baza kodu na Androida i iOS. Ekran składa się z widgetów.

Pytanie: Stateless vs Stateful?

Odpowiedź: Stateless – ekran bez własnego zmieniającego się stanu. Stateful – ma lokalny stan i `setState`. Do większej logiki lepiej Provider/Cubit/BloC.

Pytanie: Co to BuildContext?

Odpowiedź: To „adres” widgetu w drzewie. Dzięki niemu bierzesz motyw, nawigację, Provider. Po `await` sprawdzaj `context.mounted`, zanim użyjesz contextu.

Pytanie: Po co Keys?

Odpowiedź: Żeby Flutter dobrze rozpoznawał elementy listy przy zmianach kolejności i nie mieszał stanów.

Pytanie: Po co const?

Odpowiedź: Mniej zbędnego przebudowywania UI. Gdzie się da – dawaj `const`.

Pytanie: `async` / `await`?

Odpowiedź: Na rzeczy które trwają: sieć, baza. UI działa dalej. Ciężkie liczenie lepiej w osobnym `isolate` (`compute`).

Pytanie: Future vs Stream?

Odpowiedź: Future = jeden wynik. Stream = wiele wyników w czasie (np. dane z bazy na żywo).

Pytanie: Null safety?

Odpowiedź: Zmienna domyślnie nie może być null. Jak może – dajesz `?. !` używaj tylko gdy naprawdę wiesz, że nie ma null.

Pytanie: Hot reload vs hot restart?

Odpowiedź: Reload – szybka zmiana UI ze stanem. Restart – od zera. Pełny restart aplikacji przy zmianach natywnych.

Pytanie: ListView vs ListView.builder?

Odpowiedź: Zwykły ListView buduje wszystko naraz – źle przy długiej liście. Builder buduje tylko to, co widać.

Pytanie: Material vs Cupertino?

Odpowiedź: Material = styl Androida/Google. Cupertino = styl iOS. Często robimy według designu klienta.

5 Stan aplikacji: Provider, Cubit, BloC

Pytanie: Jakie znasz sposoby na stan?

Odpowiedź: setState lokalnie. Provider. Cubit. BloC. W ofercie Ideo są Cubit, Provider i BloC – o tych mów śmiało. W Fruit Trackerze używałem Providera.

Pytanie: Provider – co to?

Odpowiedź: Trzymasz stan w klasie (np. ChangeNotifier) i UI go słucha. Proste i dobre na średnie apki. Nie rób jednego wielkiego obiektu na wszystko.

Pytanie: Cubit vs BloC?

Odpowiedź: Cubit: wołasz metodę i emitujesz nowy stan – mniej kodu. BloC: wysyłasz eventy, dostajesz stany – jaśniejszy przepływ przy trudniejszej logice. Mały feature = często Cubit. Duży, skomplikowany = BloC.

Pytanie: Kiedy BloC zamiast Providera?

Odpowiedź: Gdy logika jest grubsza, chcesz stany loading/error/data, łatwe testy i czytelny przepływ. Provider wystarczy na prostszy stan UI.

Pytanie: Jak uniknąć przebudowy całego ekranu?

Odpowiedź: Słuchaj tylko potrzebnego kawałka stanu. Dziel providery/cubity. Stałe części UI dawaj jako const.

Pytanie: Gdzie robić nawigację i snackbary?

Odpowiedź: Nie w metodzie build. Lepiej w listenerze po zmianie stanu albo po skończonym Future.

6 REST – rozmowa z serwerem

Pytanie: Jak wołasz API we Flutterze?

Odpowiedź: Pakiet http albo dio. Warstwy: API client → repository → Cubit/BloC → UI. Dane z JSON mapujesz na modele.

Pytanie: Błędy sieci?

Odpowiedź: Timeout. Pokazujesz błąd w UI. Przy GET możesz spróbować ponowić. Logujesz crash/non-fatal. Offline: cache albo komunikat.

Pytanie: Logowanie / token?

Odpowiedź: Token w nagłówku. Trzymasz w secure storage, nie w zwykłych preferencjach na produkcji. Przy 401 – odśwież token albo wyloguj.

Pytanie: Paginacja?

Odpowiedź: Dociągasz kolejne strony przy scrollu. Pokazujesz loader na dole. Nie dubluj tych samych elementów.

7 Firebase

Pytanie: Czego używałeś z Firebase?

Odpowiedź: W DEVEJI Firebase był backendem pod apki mobilne (Auth, baza, ewentualnie push/crash – mów według faktów z projektu). Umiem spiąć Fluttera z Firebase na iOS i Androidzie.

Pytanie: Firestore vs Realtime Database?

Odpowiedź: Firestore = dokumenty i kolekcje (częściej dziś). Realtime = drzewo JSON, szybkie sync. Do nowych rzeczy zwykle Firestore.

Pytanie: Jak chronisz dane?

Odpowiedź: Security Rules. Użytkownik widzi tylko swoje rzeczy po `auth.uid`. Nie ufaj samym regułom po stronie apki.

Pytanie: Auth – jak działa w apce?

Odpowiedź: Logowanie (mail/Google/Apple). Słuchasz czy user jest zalogowany. Na tej podstawie pokazujesz login albo home.

Pytanie: Push (FCM)?

Odpowiedź: Token telefonu, zgody na iOS, obsługa powiadomienia gdy apka jest otwarta/w tle, przejście do właściwego ekranu.

Pytanie: Crashlytics?

Odpowiedź: Widzisz crashe z produkcji. Bez tego trudno naprawiać problemy użytkowników.

8 UI i design

Pytanie: Jak dbasz o zgodność z projektem graficznym?

Odpowiedź: Sprawdzam odstępy, fonty, kolory z Figmy. Używam theme. Porównuję na telefonie, nie tylko w emulatorze.

Pytanie: Różne rozmiary ekranów?

Odpowiedź: Flexible/Expanded, LayoutBuilder, unikanie sztywnych wysokości na ślepo.

Pytanie: Animacje?

Odpowiedź: Proste AnimatedContainer, Hero, czasem Lottie. Animacja ma pomagać, nie rozpraszać.

9 Natywne Android / iOS

Pytanie: Kiedy piszesz kod natywny przy Flutterze?

Odpowiedź: Gdy brakuje gotowego pluginu albo trzeba głęboko wejść w system (specjalne SDK, HealthKit-like, tło). Most: MethodChannel.

Pytanie: Masz doświadczenie natywne?

Odpowiedź: Tak, iOS/SwiftUI: Water Reminder, Fitness Tracker, Lotto. Android natywnie znam podstawowo – w praktyce głównie Flutter + konfiguracja projektu.

Pytanie: Różnice iOS vs Android?

Odpowiedź: Uprawnienia, push, nawigacja wstecz, zasady sklepów, różne telefony z Androidem.

10 Wrzucanie do sklepów

Pytanie: Jak wypuszczasz apkę?

Odpowiedź: Podbijasz wersję, podpisujesz, budujesz AAB/IPA, uzupełniasz opisy i privacy, testujesz na prawdziwym telefonie, wrzucasz do review / staged rollout.

Pytanie: Dlaczego App Store odrzuca?

Odpowiedź: Crash na starcie, brakujące privacy, niedziałający login, brak konta demo dla reviewera, łamanie zasad płatności.

Pytanie: AAB vs APK?

Odpowiedź: Play chce AAB. Z niego Google robi APK pod konkretne telefony.

11 Architektura i git

Pytanie: Jak dzielisz kod we Flutterze?

Odpowiedź: UI osobno, logika (Cubit) osobno, dane (API/baza) osobno. Widget nie powinien robić wszystkiego.

Pytanie: Co poprawisz w swoich starych apkach Swift?

Odpowiedź: Za dużo logiki w jednym pliku ContentView. Wydzieliłbym ViewModel i serwisy. Dodałbym testy do ważnych reguł (np. streak).

Pytanie: Git?

Odpowiedź: Branch na feature, małe PR-y, czytelne commity. Nie wrzucam haseł i `.env` do repo.

Pytanie: Testy?

Odpowiedź: W prywatnych projektach bywało słabo – wiem, że to minus. W pracy chcę testować logikę Cubitów i główne ścieżki. Umiesz powiedzieć jak przetestować prostą rzecz.

12 Twoje projekty – co mówić

12.1 DEVEJI

Pytanie: Co robiłeś?

Odpowiedź: Flutter iOS/Android, Firebase, publikacja w sklepach (VIII–X 2025). To Twój najmocniejszy punkt pod Ideo. Bez NDA: mów kategoriami zadań, nie tajnymi szczegółami klienta.

12.2 Fruit Tracker (Flutter, prywatne)

Pytanie: Co to?

Odpowiedź: Apka do liczenia owoców i warzyw. Skan ML, historia, ulubione, lista zakupów offline. Stan: Provider. Dane: SQLite na telefonie.

Pytanie: Czemu SQLite a nie tylko chmura?

Odpowiedź: Ma działać offline i trzymać dane lokalnie. Chmurę można dodać później.

Pytanie: ML – jak to działa w skrócie?

Odpowiedź: Zdjęcie → model zgaduje produkt → user potwierdza → zapis do historii. Przy słabym świetle model może się mylić, dlatego potwierdzenie.

12.3 Water Reminder

Pytanie: Co to?

Odpowiedź: Apka iOS do picia wody. Cel 4000 ml, szybkie dodawanie, streak, powiadomienia, historia. Persistence: **SwiftData** (nie Core Data).

Pytanie: Co byś poprawił?

Odpowiedź: Rozbić jeden duży ContentView na mniejsze części i osobną logikę.

12.4 Fitness Tracker

Pytanie: Co to?

Odpowiedź: Kroki z HealthKit, przybliżone kalorie (MET), waga, wykresy, profil. Mów tylko o tym, co naprawdę jest w kodzie – bez zmyślania MapKit/zdjęć jeśli ich nie ma.

12.5 MacSpaceCleaner

Pytanie: Po co o tym na rozmowie Flutter?

Odpowiedź: Pokazuje, że myślisz o architekturze i bezpieczeństwie: osobna warstwa Core/UI, reguły co wolno usuwać, potwierdzenia. To przenosi się na mobile.

12.6 Courier (Electron)

Odpowiedź: System dla firmy kurierskiej: role, zadania, tracking, PDF. Pokazuje pracę z procesem biznesowym, nie tylko ładnym UI.

12.7 Bus (Laravel)

Odpowiedź: Trasy i rozkłady, role admin/kierowca. To CRUD – bez prawdziwego GPS na żywo. Lepiej powiedzieć prawdę niż marketing.

13 Trudne pytania – uczciwe odpowiedzi

Pytanie: Na GitHubie widać głównie Swift, a oferta to Flutter?

Odpowiedź: Komercyjny Flutter i Fruit Tracker są prywatne. Publicznie pokazuję iOS i inne rzeczy. Na rozmowie mogę przejść po Flutterze, Firebase, Provider/Cubit i procesie sklepu z DEVEJI.

Pytanie: Mało testów w repo?

Odpowiedź: Tak, w projektach uczelnianych/osobistych to słaby punkt. Wiem o tym i w komercyjnym kodzie chcę to robić lepiej.

Pytanie: Nie za junior?

Odpowiedź: Mam komercyjny Flutter + sklepy + Firebase, własną apkę z ML/SQLite i mocne iOS. Szybko się uczę. Szukam miejsca, gdzie urosnę na realnych projektach.

14 Pytania behawioralne – szkielet

Pytanie: Trudny bug?

Odpowiedź: Sytuacja → co było do zrobienia → co zrobiłeś (reprodukcja, logi, fix) → efekt. Wypełnij prawdziwym przykładem z DEVEJI.

Pytanie: Deadline vs jakość?

Odpowiedź: Tniesz zakres, nie fundamenty (crash, bezpieczeństwo). Trade-off mówisz wcześniej.

Pytanie: Z czego jesteś dumny?

Odpowiedź: Wybierz jedno: publikacja w sklepie / Fruit Tracker / dopracowane powiadomienia w Water Reminder / architektura MacSpaceCleaner.

15 Gdyby jednak dali małe zadanie

Pytanie: Model z JSON?

Odpowiedź: Klasa z polami + `fromJson`.

Pytanie: Lista z API?

Odpowiedź: Stan loading → pobranie → success/error. UI: `ListView.builder`.

Pytanie: `FutureBuilder` czy `Cubit`?

Odpowiedź: `FutureBuilder` na prostą, jednorazową rzecz. `Cubit` gdy stan wraca na wielu ekranach, chcesz retry i testy.

Pytanie: Szukajka z opóźnieniem?

Odpowiedź: Poczekać ~300 ms po wpisaniu zanim wyślesz request. Anuluj poprzednie zapytanie.

Pytanie: `Context` po `await`?

Odpowiedź: Sprawdź `context.mounted`. Inaczej możesz dostać błąd po wyjściu z ekranu.

16 Pytania, które Ty zadajesz Im

1. Jaki stan używacie na co dzień: `BloC`, `Cubit` czy `Provider`?
2. Jak wygląda code review i wrzucanie do sklepów?
3. Nad czym teraz pracuje zespół Flutter?
4. Jak wygląda okres próbny / mentoring?

5. Ile jest natywnego Swift/Kotlin obok Fluttera?
6. Ile dni w biurze na start?
7. Jakie są kolejne etapy rekrutacji?

17 Checklista przed rozmową

- 2–3 zdania o DEVEJI, Fruit Tracker, Water Reminder.
- Cubit vs BloC vs Provider – umiesz powiedzieć własnymi słowami.
- Firebase Auth + Firestore – krótko.
- Nie ściemniaj względem kodu (SwiftData, brak MapKit jeśli go nie ma).
- 3 pytania do nich.
- Adres: Nad Przyrwą 13, 10:00, ok. 45 minut.

Mów prosto, konkretami z projektów. Jak czegoś nie wiesz – powiedz jak byś to sprawdził. Powodzenia.